



中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

并行程序设计 with 算法 (实验)

10-CUDA并行矩阵乘法

吴迪、刘学正

中山大学 计算机学院

◉ 实验内容

- 实现基础CUDA矩阵乘法
- 优化矩阵乘法：共享内存分块，寄存器分块
- 测量不同实现的运行时间

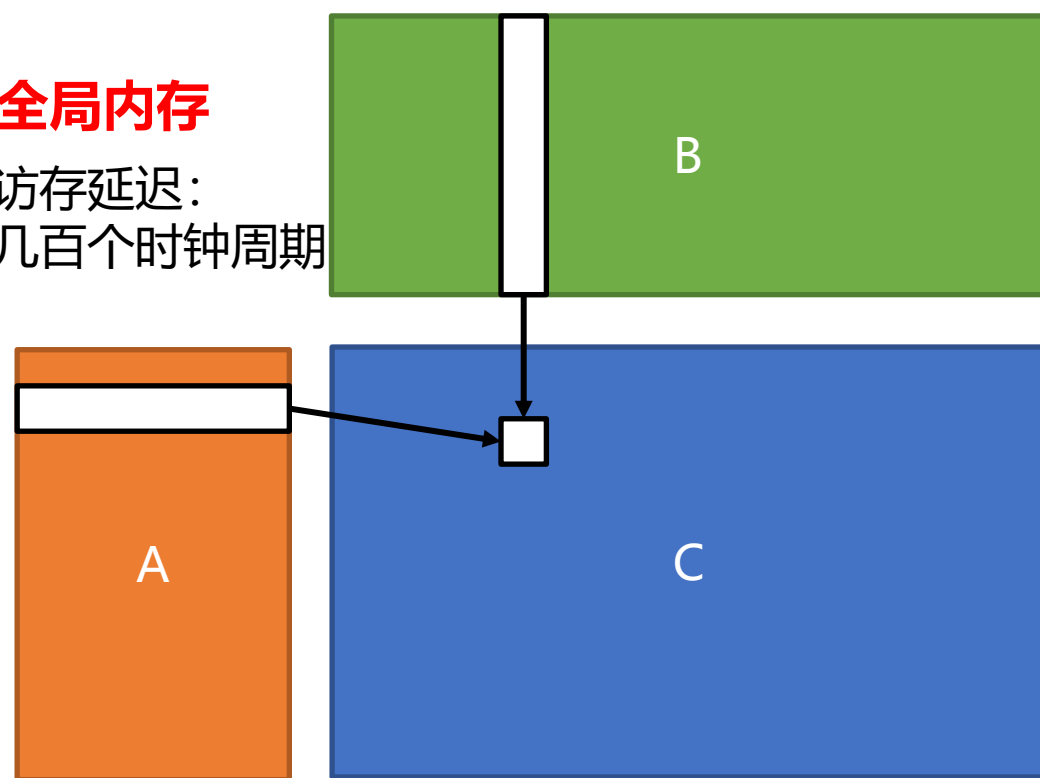
◉ 实验目的

- 理解CUDA编程模型（Grid、Block、Thread）及其在矩阵乘法中的应用
- 学习GPU内存层次结构（全局内存、共享内存、寄存器）以及分块优化、寄存器复用等核心优化策略

- **原理：** 每个GPU线程计算输出矩阵的一个元素，通过遍历输入矩阵的行和列进行乘加运算，最终将结果写入全局内存
 - **延迟高和带宽受限：** 全局内存访问需数百周期、每个线程均要读写全局内存
 - **带宽浪费：** A (B) 的同一行 (列) 被多个线程重复读取

全局内存

访存延迟：
几百个时钟周期

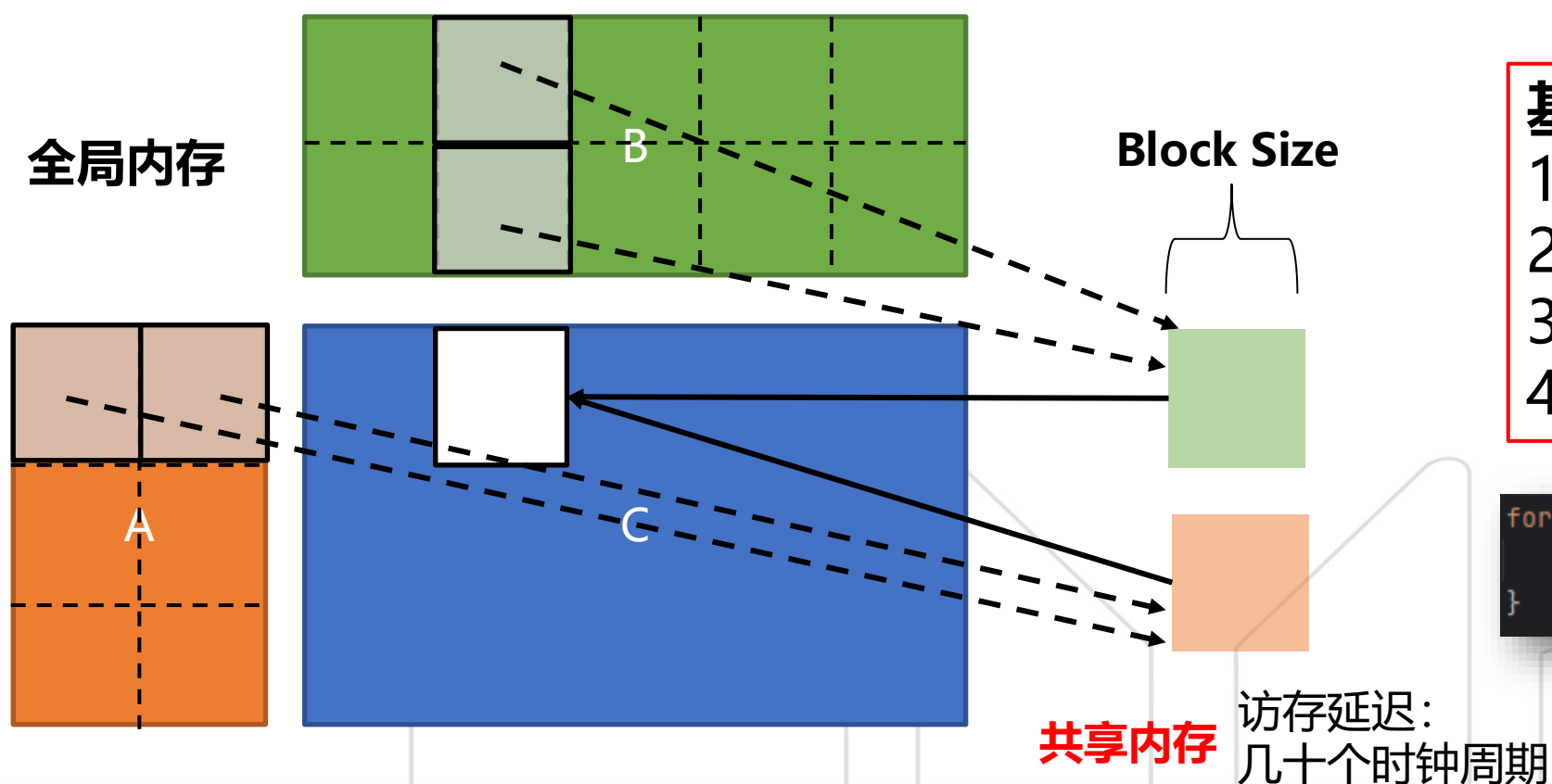


基本步骤：

1. 从全局内存读取A的一行
2. 从全局内存读取B的一列
3. 计算一次乘累加得到C的一个元素，并写回全局内存

```
__global__ void matrixMulKernelNaive(const float *d_A, const float *d_B, float *d_C, int m, int n, int k) {  
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;  
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (row < m && col < k) {  
        float sum = 0.0f;  
        for (int i = 0; i < n; i++) {  
            sum += d_A[row * n + i] * d_B[i * k + col];  
        }  
        d_C[row * k + col] = sum;  
    }  
}
```

- **原理：** 将频繁访问的全局内存数据批量加载到速度更快的共享内存中。通过分块计算实现数据重用，减少全局内存访问次数
 - **分块思想：** 将大矩阵划分为数据块，每个线程块负责处理一个数据块
 - **数据重用：** 一个数据块被加载到共享内存后，该线程块内的所有线程都可以重复读取它，实现“一次加载，多次计算”



基本步骤：

1. 读取A矩阵一个block到共享内存
2. 读取B矩阵一个block到共享内存
3. 利用共享内存数据进行乘加运算
4. 写回一个block到全局内存

```
for (int j = 0; j < blockDim.x; j++) {  
    sum += s_A[ty * blockDim.x + j] * s_B[j * blockDim.y + tx];  
}
```

- 原理：在共享内存分块的基础上，让每个线程利用寄存器缓存数据，避免对共享内存的重复访问，进一步提升计算访存比
 - 分块计算：每个线程内部处理一个tile，提高计算密度
 - 寄存器缓存：线程将频繁使用的数据存储在寄存器中，避免重复加载共享内存

基本步骤：

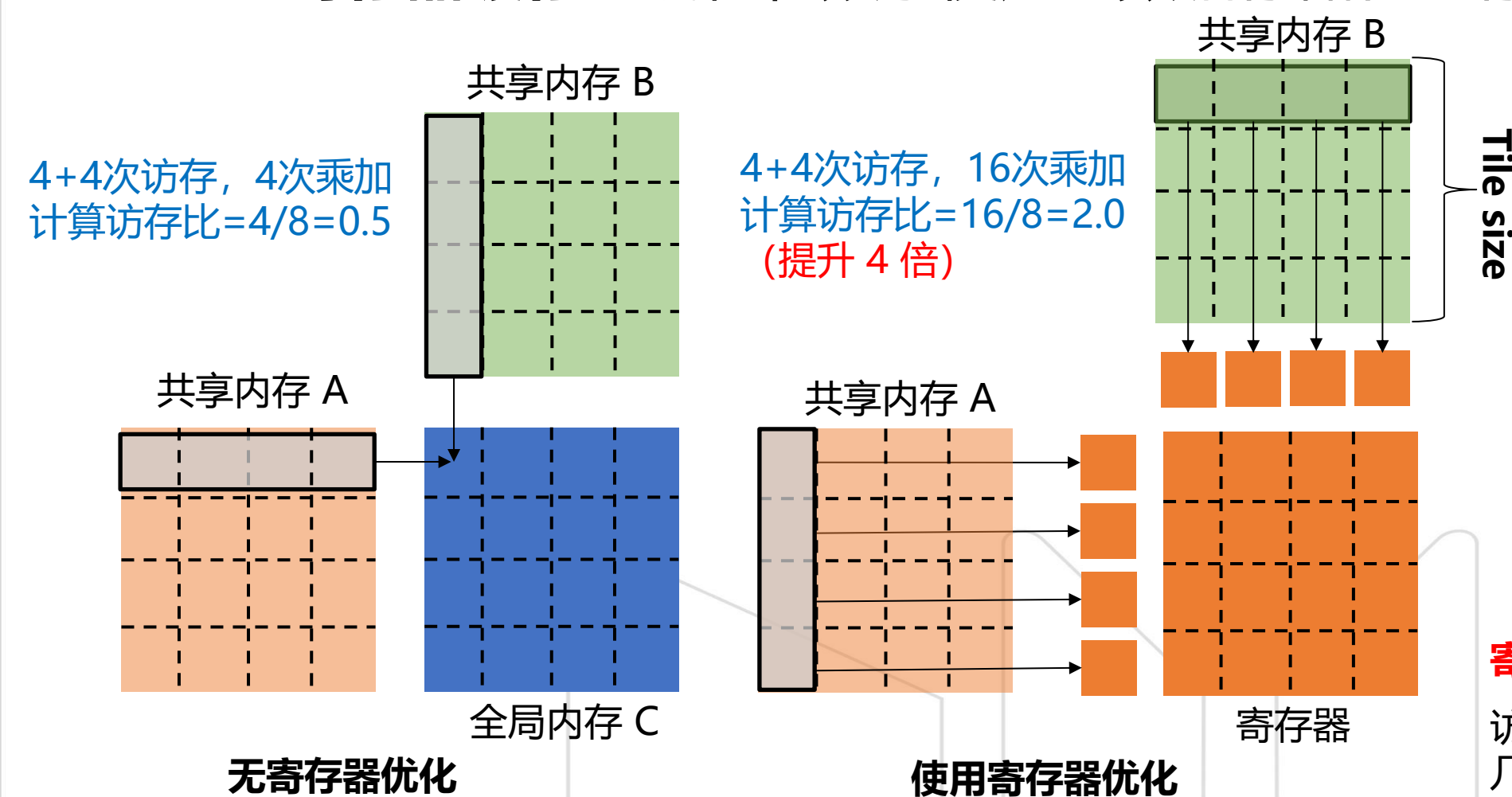
1. 从共享内存A加载tile_size $\times$ 1的A块到寄存器
2. 从共享内存B加载1 $\times$ tile_size的B块到寄存器
3. 利用寄存器进行多次乘加，累加结果留在寄存器中
4. 遍历完tile后，将寄存器累加的一整组结果写回全局内存

提示

- Tile Size 决定单个线程工作量/寄存器个数
- Block Size 决定并行度/共享内存的分块大小

寄存器

访存延迟：
几个时钟周期



◉ 要求

- 使用CUDA实现并行通用矩阵乘法
- 分析不同因素性能的影响
 - 线程块大小、矩阵规模：如何提高占用率？
 - 访存方式：何时使用何种存储？
 - 数据/任务划分方式：按行，列，数据块划分等

Questions?

